```
# python3
Python 3.6.8 (default, Jan 14 2019, 11:02:34)
[GCC 8.0.1 20180414 (experimental) [trunk revision 259383]] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 5/2
2.5
>>> exit()
# python2
Python 2.7.15+ (default, Nov 27 2018, 23:36:35)
[GCC 7.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 5/2
2
>>> 5.0/2.0
2.5
>>> 5.0/2
2.5
>>> 5/2.0
2.5
>>> exit()
```

Figure 3: Differences in the division operator

```
RefactoringTool: Refactored py3.py
--- py3.py      (original)
+++ py3.py      (refactored)
@@ -1,3 +1,3 @@
 num = 5/2
-print num
+print(num)

RefactoringTool: Files that were modified:
RefactoringTool: py3.py
# cat py3.py
num = 5/2
print(num)

# python3 py3.py
2.5
```

Figure 4: Conversion from Python 2 to Python 3

The division operator

The division operator '/' results in integer division in Python 2, whereas in Python 3, the division operator performs normal division. For example, when the operation '5/2' is performed in Python 3 we get 2.5 as the result. But in Python 2, we get 2 as our result. Figure 3 shows this difference when executed on Python 3 and Python 2 shells. Figure 3 also shows us that the operations '5.0/2.0', '5.0/2', and '5/2.0' will result in 2.5 as the output in Python 2 also.

ASCII strings in Python 2

In Python 3, strings are stored as Unicode by default, whereas in Python 2, we need to prefix a string with a 'u' to make it a Unicode string. In Python 2, strings are stored as ASCII, by default. This support for Unicode by Python 3 is really important because Unicode supports a far richer character set when compared with ASCII.

Functions that are not supported by Python 3

The function *xrange( )* supported by Python 2 is not supported in Python 3. The *range( )* function in Python 3 behaves similar to the *xrange( )* function in Python 2. Similarly, the function *raw_input( )* supported by Python 2 is not supported by Python 3. In Python 3, we have to use the function *input( )*. Remember the use of these deprecated functions in Python 3 will lead to an error.

The differences pointed out here are just a series of one-liners and do not represent all the differences between the two versions, but if somebody could build on from this article, then all the differences between Python 2 and Python 3 can be fully enumerated to benefit the Python programming community. But even if you are planning to stick with Python 2, you don't need to worry much. *2to3* is a Python program that reads Python 2 source code and converts it into Python 3 source code. A standard library called *lib2to3*, which contains a lot of fixes, is used to do this conversion.

The *lib2to3* library is flexible and generic, thus allowing us to write our own fixes for *2to3* so that the unknown differences can also be handled. Thus, a complete list of differences between Python 2 and Python 3 comes very handy in this situation. The URL *https://docs.python.org/2/library/2to3.html* will take us to the documentation of *2to3*. This page also mentions many other differences between Python 2 and Python 3 not discussed in this article. There are some online converters also — for converting a Python 2 script to Python 3. But a programmer should be very careful while using any such tool. Consider the Python 2 script called *py3.py*.

```
num = 5/2
print num
```

On execution, the Python 2 script *py3.py* prints 2 as the output. Figure 4 shows the use of *2to3* to convert the Python 2 script *py3.py* to Python 3. The command *2to3 -w py3.py* converts the Python 2 script *py3.py* to Python 3. From the figure, it is clear that the print statement of Python 2 is properly converted to a print function whereas the integer division operator '/' is left unchanged. Figure 4 also shows the output of the converted Python 3 script is different from the output of the original Python 2 script. The Python 3 script prints 2.5 as output. This is not at all acceptable. The output of the script should not change when converted from Python 2 to Python 3. Thus the blind use of a tool to convert a Python 2 script to Python 3 is not at all advisable because it might lead to potential bugs in the Python 3 script. This is why a judicious use of such conversion tools and an in-depth knowledge of the differences between Python 2 and Python 3 are necessary. An expanded list like the one we have discussed here will be very useful in such situations. END

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. The author maintains a technical blog at *www.computingforbeginners.blogspot.in.*